

بسمه تعالی

درس سیستم های نهفته - نیمسال تحصیلی ۰۴-۰۵

دانشگاه صنعتی شریف - دانشکده مهندسی برق



تمرین سری چهارم

امیرحسین مطیعی - شماره دانشجویی ۴۰۱۱۰۲۵۵۳

(۱) کتابخانه Mongoose در c/c++

کتابخانه Mongoose چیست؟

Mongoose یک کتابخانه سبک شبکه برای زبان های C و C++ است که امکان ایجاد سرورهای وب و برنامه های شبکه ای را فراهم می کند. این کتابخانه رابط های برنامه نویسی غیرمسدودکننده و رویدادمحور برای پروتکل هایی مانند TCP, UDP, HTTP, HTTPS, WebSocket و MQTT ارائه می دهد. به همین دلیل، هم در رایانه های معمولی با سیستم عامل هایی مانند Windows و Linux و هم در تجهیزات توکار و میکروکنترلرهایی مانند STM32 و ESP32 قابل استفاده است. ساختار رویدادمحور به این معناست که برنامه به جای متوقف شدن و انتظار برای دریافت درخواست، به کار خود ادامه می دهد و هر زمان رویدادی مانند اتصال کاربر، دریافت درخواست HTTP یا قطع ارتباط رخ دهد، یک تابع مشخص برای رسیدگی به آن اجرا می شود. این ویژگی باعث می شود برنامه بتواند چندین اتصال شبکه را با مصرف مناسب منابع مدیریت کند.

کاربردهای کتابخانه Mongoose

یکی از اصلی ترین کاربردهای Mongoose، اضافه کردن یک HTTP Server به برنامه های نوشته شده با C یا C++ است. برای مثال، می توان در یک دستگاه صنعتی، برد الکترونیکی یا نرم افزار دسکتاپ، صفحه ای تحت وب ایجاد کرد تا کاربر از طریق مرورگر وضعیت سیستم را مشاهده کرده یا تنظیمات آن را تغییر دهد. این کتابخانه همچنین برای طراحی REST API کاربرد دارد. در این حالت، برنامه مسیریایی مانند /api/status، /api/login یا /api/device تعریف می کند و نرم افزارهای دیگر می توانند با ارسال درخواست های GET، POST، PUT یا DELETE به این مسیرها، اطلاعات دریافت کنند یا عملیاتی را انجام دهند. پاسخ API معمولاً به شکل JSON ارسال می شود.

Mongoose می تواند فایل های ثابت مانند HTML، CSS، JavaScript و تصاویر را نیز در اختیار مرورگر قرار دهد. بنابراین، می توان از آن برای ساخت یک سرور ترکیبی استفاده کرد؛ به گونه ای که فایل های رابط کاربری را نمایش دهد و هم زمان درخواست های API را پردازش کند. مستندات رسمی نیز این روش ترکیبی را برای ساخت رابط وب تجهیزات معرفی کرده اند.

از دیگر کاربردهای آن می توان به ایجاد داشبورد مدیریتی تجهیزات، برقراری ارتباط بلادرنگ با WebSocket، اتصال دستگاه های اینترنت اشیا به سرویس های MQTT، انتقال امن اطلاعات با HTTPS و به روزرسانی نرم افزار تجهیزات از راه دور اشاره کرد.

پیاده‌سازی HTTP Server با Mongoose

برای استفاده از این کتابخانه، معمولاً دو فایل `mongoose.c` و `mongoose.h` به پروژه C یا C++ اضافه می‌شوند. سپس فایل سرآیند در برنامه قرار می‌گیرد:

```
#include "mongoose.h"
```

پس از آن، یک مدیر رویداد از نوع `mg_mgr` ساخته می‌شود. این مدیر وظیفه نگهداری و مدیریت اتصال‌های شبکه را بر عهده دارد. تابع `mg_mgr_init` آن را مقداردهی اولیه می‌کند و تابع `mg_http_listen` یک سرور HTTP را روی آدرس و پورت موردنظر فعال می‌سازد. برنامه نیز باید تابع `mg_mgr_poll` را به‌صورت پیوسته اجرا کند تا درخواست‌های شبکه دریافت و پردازش شوند.

ساختار کلی یک سرور ساده به صورت زیر است:

```
#include "mongoose.h"
static void handle_request(struct mg_connection *connection,
                          int event,
                          void *event_data) {
    if (event == MG_EV_HTTP_MSG) {
        struct mg_http_message *request =
            (struct mg_http_message *) event_data;

        mg_http_reply(
            connection,
            200,
            "Content-Type: text/plain\r\n",
            "Hello from Mongoose\n"
        );
    }
}

int main(void) {
    struct mg_mgr manager;

    mg_mgr_init(&manager);

    mg_http_listen(
        &manager,
        "http://0.0.0.0:8000",
        handle_request,
        NULL
    );

    while (1) {
        mg_mgr_poll(&manager, 100);
    }

    mg_mgr_free(&manager);
    return 0;
}
```

در این برنامه، سرور روی پورت 8000 فعال می‌شود. هنگامی که مرورگر یا نرم‌افزار دیگری یک درخواست HTTP ارسال کند، رویداد `MG_EV_HTTP_MSG` ایجاد شده و تابع `handle_request` اجرا می‌شود. تابع `mg_http_reply` نیز کد وضعیت، سرآیندها و متن پاسخ را برای کاربر ارسال می‌کند.

پیاده‌سازی API با Mongoose

برای ایجاد API باید آدرس درخواست دریافت شده بررسی شود و برای هر مسیر پاسخ مناسبی در نظر گرفته شود. برای نمونه، مسیر `/api/status` می‌تواند وضعیت سیستم را به شکل JSON برگرداند:

```
static void handle_request(struct mg_connection *connection,
                          int event,
                          void *event_data) {
    if (event == MG_EV_HTTP_MSG) {
        struct mg_http_message *request =
            (struct mg_http_message *) event_data;

        if (mg_match(request->uri, mg_str("/api/status"), NULL)) {
            mg_http_reply(
                connection,
                200,
                "Content-Type: application/json\r\n",
                "{\"status\":\"running\",\"temperature\":25}\n"
            );
        } else {
            mg_http_reply(
                connection,
                404,
                "Content-Type: application/json\r\n",
                "{\"error\":\"route not found\"}\n"
            );
        }
    }
}
```

در این مثال، ابتدا آدرس درخواست با کمک تابع `mg_match` بررسی می‌شود. اگر کاربر مسیر `/api/status` را درخواست کرده باشد، سرور اطلاعات وضعیت را با کد 200 و قالب JSON ارسال می‌کند. اگر مسیر شناخته‌شده نباشد، کد خطای 404 بازگردانده می‌شود. در API های پیشرفته‌تر می‌توان بدنه درخواست‌های POST را خواند، داده‌های JSON را استخراج کرد، اطلاعات را در پایگاه داده ذخیره کرد و نتیجه عملیات را برای کاربر فرستاد. Mongoose ابزارهایی برای تحلیل درخواست HTTP، خواندن بدنه پیام و پردازش JSON در اختیار برنامه‌نویس قرار می‌دهد.

۲) پایگاه داده های توزیع

سیستم های توسعه یافته برای پایگاه داده های توزیع شده

پایگاه داده توزیع شده، پایگاه داده ای است که اطلاعات آن به جای ذخیره شدن روی یک رایانه یا سرور، میان چندین سرور یا گره شبکه توزیع می شود. این سرورها با یکدیگر همکاری می کنند و از دید کاربران مانند یک پایگاه داده واحد عمل می کنند. سیستم های مختلفی برای مدیریت این نوع پایگاه داده توسعه یافته اند که برخی از آن ها از مدل NoSQL و برخی دیگر از مدل رابطه ای و زبان SQL استفاده می کنند.

نمونه سیستم ها و کاربرد هر کدام

Apache Cassandra

یک پایگاه داده توزیع شده NoSQL است که برای ذخیره حجم بسیار زیادی از اطلاعات و ارائه دسترس پذیری بالا طراحی شده است. این سیستم برای شبکه های اجتماعی، سامانه های پیام رسان، اینترنت اشیا و برنامه هایی با تعداد زیاد درخواست مناسب است.

MongoDB

یک پایگاه داده سندگراست که اطلاعات را به شکل اسناد انعطاف پذیر ذخیره می کند. MongoDB با استفاده از شاردینگ، داده ها را میان چند سرور تقسیم می کند و برای وبسایت ها، فروشگاه های اینترنتی، سامانه های مدیریت محتوا و برنامه هایی که ساختار داده آن ها مرتب تغییر می کند کاربرد دارد.

Amazon DynamoDB

یک پایگاه داده NoSQL توزیع شده، بدون سرور و کاملاً مدیریت شده در سرویس ابری آمازون است. از آن برای برنامه های اینترنتی، بازی های آنلاین، فروشگاه های بزرگ و سامانه هایی استفاده می شود که به سرعت پاسخ گویی و مقیاس پذیری بالا نیاز دارند.

Google Cloud Spanner

یک پایگاه داده توزیع شده و مدیریت شده است که از تراکنش ها و زبان SQL پشتیبانی می کند و می تواند اطلاعات را در چند منطقه جغرافیایی نگهداری کند. این سیستم برای سامانه های مالی، تجاری و برنامه های جهانی که به سازگاری دقیق اطلاعات نیاز دارند مناسب است.

CockroachDB

یک پایگاه داده SQL توزیع شده است که اطلاعات و عملیات را میان چندین گره تقسیم می کند. این سیستم در برابر خرابی سرورها و حتی مراکز داده مقاوم است و برای برنامه های ابری، سامانه های بانکی و خدماتی که نباید با خرابی یک سرور متوقف شوند کاربرد دارد.

۳) API

API چیست و چگونه کار می کند؟

API مخفف عبارت Application Programming Interface و به معنی «رابط برنامه نویسی کاربردی است. API مجموعه ای از قوانین، روش ها و مسیرهای مشخص است که به نرم افزارها اجازه می دهد بدون نیاز به آگاهی از جزئیات داخلی یکدیگر، با هم ارتباط برقرار کرده و اطلاعات یا خدمات مورد نیاز را مبادله کنند. به زبان ساده، API مانند یک واسطه یا پیشخدمت در رستوران عمل می کند؛ کاربر سفارش خود را به پیشخدمت می دهد، پیشخدمت آن را به آشپزخانه منتقل می کند و سپس نتیجه را به کاربر تحویل می دهد. در این مثال، نرم افزار درخواست کننده نقش مشتری، API نقش پیشخدمت و نرم افزار یا سرور ارائه دهنده خدمت نقش آشپزخانه را دارد.

هنگامی که یک نرم افزار به اطلاعات یا امکانات نرم افزار دیگری نیاز داشته باشد، درخواستی را از طریق API ارسال می کند. این درخواست معمولاً شامل آدرس مشخصی به نام Endpoint، نوع عملیات، اطلاعات ورودی و در برخی موارد کلید یا توکن امنیتی است. سرور پس از دریافت و بررسی درخواست، نتیجه را در قالب یک پاسخ بازمی گرداند. این پاسخ معمولاً شامل داده ها، وضعیت موفق یا ناموفق بودن عملیات و پیام های احتمالی خطاست. اطلاعات در API های اینترنتی اغلب با قالب JSON منتقل می شوند، زیرا این قالب ساده، کم حجم و قابل فهم برای بیشتر زبان های برنامه نویسی است.

کاربرد های API

برای نمونه، یک برنامه هواشناسی لازم نیست خودش تجهیزات اندازه گیری دما و بارندگی داشته باشد. این برنامه می تواند از API یک سرویس هواشناسی استفاده کند، نام شهر را ارسال کند و اطلاعاتی مانند دما، رطوبت و وضعیت بارندگی را دریافت و نمایش دهد. همچنین در یک فروشگاه اینترنتی، هنگام پرداخت هزینه، سایت فروشگاه از طریق API به درگاه بانکی متصل می شود. فروشگاه مبلغ و اطلاعات تراکنش را ارسال می کند و درگاه بانکی نتیجه پرداخت را به سایت بازمی گرداند. ورود به یک برنامه با حساب گوگل، نمایش موقعیت روی نقشه، ارسال پیامک، محاسبه هزینه ارسال کالا و دریافت قیمت ارز نیز نمونه هایی از کاربرد API هستند.

نقش API در ارتباط میان نرم افزارها و سرویس ها

API باعث می شود نرم افزارها به صورت استاندارد، سریع و ایمن با یکدیگر ارتباط برقرار کنند. برنامه نویسان به جای آنکه تمام امکانات مورد نیاز را از ابتدا طراحی کنند، می توانند از خدمات آماده شرکت ها و سامانه های دیگر استفاده کنند. این موضوع باعث کاهش زمان و هزینه توسعه، جلوگیری از دوباره کاری و افزایش سرعت ساخت نرم افزار می شود. همچنین API امکان تفکیک بخش های مختلف یک سامانه را فراهم می کند؛ برای مثال، بخش ظاهری یک سایت می تواند از طریق API با بخش مدیریت اطلاعات و پایگاه داده ارتباط داشته باشد.

بسیاری از API ها برای حفظ امنیت از روش های احراز هویت مانند API Key، نام کاربری و رمز عبور یا Access Token استفاده می کنند. این اطلاعات مشخص می کنند چه شخص یا نرم افزاری درخواست را ارسال کرده و چه مجوزهایی دارد. همچنین ممکن است برای تعداد درخواست ها محدودیت تعیین شود تا از فشار بیش از حد به سرور یا سوء استفاده از خدمات جلوگیری شود. در صورتی که درخواست اشتباه باشد یا کاربر مجوز لازم را نداشته باشد، API کد خطای مناسبی مانند ۴۰۰ برای درخواست نامعتبر، ۴۰۱ برای نداشتن احراز هویت، ۴۰۴ برای پیدا نشدن منبع و ۵۰۰ برای خطای داخلی سرور ارسال می کند.

۴) شبکه محلی یا LAN

شبکه محلی یا LAN چیست؟

شبکه محلی یا LAN که مخفف عبارت Local Area Network است، به شبکه‌ای گفته می‌شود که تعدادی رایانه، تلفن همراه، چاپگر، سرور و تجهیزات دیگر را در یک محدوده جغرافیایی کوچک مانند خانه، مدرسه، اداره، شرکت یا آزمایشگاه به یکدیگر متصل می‌کند. هدف اصلی از ایجاد شبکه محلی، فراهم کردن امکان تبادل اطلاعات و استفاده مشترک از منابع و خدمات است.

کاربردهای شبکه محلی

در یک شبکه LAN، کاربران می‌توانند فایل‌ها و اطلاعات را با یکدیگر به اشتراک بگذارند، از چاپگر مشترک استفاده کنند، به یک سرور مرکزی متصل شوند و از طریق یک اتصال مشترک به اینترنت دسترسی داشته باشند. شبکه‌های محلی معمولاً سرعت بالایی دارند و مدیریت آن‌ها نسبت به شبکه‌های گسترده ساده‌تر است.

نحوه برقراری ارتباط نودها در شبکه LAN

هر وسیله‌ای که به شبکه متصل شود، یک نود یا گره شبکه نامیده می‌شود. نودها می‌توانند از طریق کابل شبکه یا اتصال بی‌سیم به شبکه متصل شوند. در شبکه‌های کابلی معمولاً از کابل اترنت و دستگاهی به نام سوئیچ استفاده می‌شود. سوئیچ نودها را به یکدیگر متصل کرده و اطلاعات را به دستگاه مقصد هدایت می‌کند. در شبکه‌های بی‌سیم نیز تجهیزات از طریق Wi-Fi و دستگاهی مانند مودم یا Access Point با یکدیگر ارتباط برقرار می‌کنند. برای اینکه نودها بتوانند یکدیگر را شناسایی کنند، هر دستگاه دارای یک آدرس مشخص مانند آدرس IP و آدرس MAC است. هنگامی که یک نود قصد ارسال اطلاعات به نود دیگری را دارد، داده‌ها را به بسته‌های کوچک تقسیم کرده و آدرس مقصد را به آن‌ها اضافه می‌کند. سوئیچ یا تجهیزات شبکه این بسته‌ها را دریافت کرده و به دستگاه موردنظر ارسال می‌کنند. دستگاه مقصد نیز بسته‌ها را دریافت و اطلاعات اصلی را بازسازی می‌کند. در صورتی که دو نود در یک شبکه محلی باشند، می‌توانند بدون عبور از اینترنت مستقیماً با یکدیگر ارتباط برقرار کنند. اما برای ارتباط با شبکه‌های دیگر یا اینترنت، معمولاً از دستگاهی به نام روتر استفاده می‌شود. روتر وظیفه هدایت اطلاعات میان شبکه محلی و شبکه‌های خارجی را بر عهده دارد.

۵) سیستم های توزیع شده

سیستم های توزیع شده چیستند؟

سیستم توزیع شده به مجموعه ای از چند رایانه، سرور یا دستگاه مستقل گفته می شود که از طریق شبکه با یکدیگر ارتباط برقرار می کنند و برای انجام یک هدف مشترک همکاری می کنند. این دستگاه ها ممکن است از نظر فیزیکی در مکان های مختلف قرار داشته باشند، اما از دید کاربر مانند یک سیستم واحد عمل می کنند. برای مثال، سرویس های ابری، موتورهای جستجو، شبکه های بانکی، فروشگاه های اینترنتی و سامانه های پیام رسان از سیستم های توزیع شده استفاده می کنند. در چنین سیستمی، پردازش اطلاعات و ذخیره سازی داده ها تنها بر عهده یک رایانه نیست، بلکه وظایف میان چندین گره یا Node تقسیم می شوند. هر گره می تواند بخشی از محاسبات را انجام دهد، اطلاعاتی را ذخیره کند یا خدمات مشخصی ارائه دهد. این گره ها از طریق شبکه پیام هایی برای یکدیگر ارسال می کنند و نتیجه فعالیت خود را با سایر بخش های سیستم هماهنگ می سازند.

ویژگی های سیستم های توزیع شده

یکی از مهم ترین ویژگی های سیستم های توزیع شده، استفاده از چندین گره مستقل است. هر گره منابع پردازشی، حافظه و سیستم عامل مخصوص خود را دارد، اما برای انجام عملیات با گره های دیگر همکاری می کند. ارتباط میان این گره ها از طریق شبکه انجام می شود و معمولاً حافظه یا ساعت مشترک دقیقی میان آن ها وجود ندارد. ویژگی دیگر این سیستم ها، شفافیت است. شفافیت به این معناست که کاربر لازم نیست بداند اطلاعات در کدام سرور ذخیره شده یا عملیات توسط کدام گره انجام می شود. کاربر تنها نتیجه نهایی را مشاهده می کند و پیچیدگی های داخلی سیستم از او پنهان می ماند. سیستم های توزیع شده قابلیت توسعه پذیری دارند. یعنی با افزایش تعداد کاربران یا حجم اطلاعات، می توان گره ها و سرورهای جدیدی به سیستم اضافه کرد. همچنین در بسیاری از این سیستم ها داده ها در چند مکان نگهداری می شوند تا در صورت از کار افتادن یک گره، اطلاعات همچنان از طریق گره های دیگر در دسترس باشند.

مزایای سیستم های توزیع شده

یکی از مهم ترین مزایای سیستم های توزیع شده، افزایش قدرت پردازش است. با تقسیم یک کار بزرگ میان چندین رایانه، عملیات می تواند سریع تر انجام شود. همچنین چندین کاربر می توانند به طور هم زمان از منابع و خدمات سیستم استفاده کنند. مزیت دیگر، افزایش قابلیت اطمینان و دسترس پذیری است. اگر یکی از گره ها دچار خرابی شود، سایر گره ها می توانند به فعالیت خود ادامه دهند و خدمات سیستم به طور کامل متوقف نشود. برای دستیابی به این هدف، معمولاً از نسخه های تکراری داده ها و سرورهای جایگزین استفاده می شود. این سیستم ها امکان اشتراک گذاری منابع را نیز فراهم می کنند. کاربران و برنامه ها می توانند از اطلاعات، پردازنده ها، فضای ذخیره سازی و خدمات مشترک استفاده کنند. علاوه بر این، توسعه سیستم معمولاً ساده تر است؛ زیرا به جای تعویض یک سرور مرکزی بزرگ، می توان سرورهای بیشتری به مجموعه اضافه کرد. از دیگر مزایای سیستم های توزیع شده می توان به کاهش زمان پاسخ گویی، توزیع بار کاری میان سرورها، ارائه خدمات به کاربران در مناطق جغرافیایی مختلف و کاهش وابستگی به یک رایانه مرکزی اشاره کرد.

چالش های طراحی سیستم های توزیع شده

طراحی سیستم های توزیع شده با چالش های متعددی همراه است. یکی از مهم ترین چالش ها، هماهنگ سازی گره ها است. از آنجا که گره ها به صورت مستقل فعالیت می کنند و ساعت مشترک دقیقی ندارند، تعیین ترتیب صحیح رویدادها و هماهنگی عملیات میان آن ها دشوار است. چالش دیگر، تأخیر و خرابی شبکه است. پیام ها ممکن است با تأخیر به مقصد برسند، گم شوند یا چند بار ارسال شوند. همچنین ممکن است ارتباط یک گره با سایر بخش های سیستم قطع شود، در حالی که خود گره همچنان فعال باشد. بنابراین سیستم باید بتواند

چنین شرایطی را تشخیص دهد و واکنش مناسبی نشان دهد. حفظ سازگاری داده ها نیز یکی از مسائل مهم است. هنگامی که چند نسخه از یک داده در سرورهای مختلف وجود داشته باشد، تغییر یک نسخه باید به سایر نسخه ها نیز منتقل شود. اگر این تغییر به موقع انجام نشود، کاربران ممکن است اطلاعات متفاوت یا قدیمی دریافت کنند. امنیت نیز در سیستم های توزیع شده اهمیت زیادی دارد؛ زیرا اطلاعات از طریق شبکه منتقل می شوند و گره های متعددی به سیستم دسترسی دارند. به همین دلیل، استفاده از احراز هویت، کنترل دسترسی، رمزنگاری اطلاعات و محافظت در برابر حملات ضروری است. از دیگر چالش ها می توان به مدیریت خرابی گره ها، تقسیم مناسب بار کاری، جلوگیری از ایجاد گلوگاه، حفظ حریم خصوصی، اشکال زدایی دشوار و افزایش پیچیدگی طراحی و نگهداری سیستم اشاره کرد.